Chanakya WhatsApp Bot

Go-Live Runbook — Exact Steps to Launch

Prepared for: Sriram | Project: chanakya-bot | Status: code complete, hardened, and committed — deployment pending

This document is your step-by-step checklist to take the bot from “code finished on my laptop” to “live and answering real customers on WhatsApp.” Every step below is something **you** need to do — account setup, clicking buttons in Meta/Railway/GitHub, or copying a value into a settings screen. The code itself is already built, security-hardened, and tested. Follow the steps in order; each one notes whether it costs money, and why it matters.

Symbol	Meaning
PAID	This step involves a real cost
FREE	No cost for this step
CRITICAL	Skipping this creates a real security or reliability risk

Phase	What happens	Time
A. Secure the code	Push to GitHub, rotate every exposed secret	30 min
B. Get real credentials	Permanent Meta token, app secret, business verification	20-45 min (1)
C. Deploy	Railway hosting, environment variables, persistent storage	30 min
D. Connect Meta	Point the webhook at your live URL, verify	10 min
E. Sheet + templates	Header refresh, optional pickup template	15 min + (2)
F. Smoke test	Full walk-through on the real number	10 min
G. Go live	Publish the number to customers	—

(1) Business verification can take Meta 1-3 business days to review — start this early. (2) Template approval is typically minutes to a few hours.

Phase A — Secure the Code

Before anything goes live, two things need to happen: your code needs to be safely backed up (GitHub), and every credential that may have been exposed needs to be rotated (replaced with a new one).

STEP 1 — Push the local commits to GitHub — FREE

The project is already a Git repository on your machine with two commits made. GitHub Desktop is open with the repo loaded.

- Open **GitHub Desktop** (already pointed at the `chanakya-bot` repo).
- Click **"Publish repository."**
- In the dialog: **tick "Keep this code private."** This repo must never be public — it documents your architecture and, while secrets are excluded, there's no reason to expose it.
- Click **Publish repository**.
- For any future change: write a short summary > **Commit to main** > **Push origin**.

Why: *this is your off-machine backup and the thing Railway/Render will deploy from directly.*

STEP 2 — Rotate every exposed credential — CRITICAL

During development, a full copy of your `.env` file (containing live secrets) was bundled into `chanakya-bot.zip`, and your ngrok tunnel token was pasted in plaintext into a setup notes file. Treat every one of the following as potentially seen by someone else, even if you're fairly sure it wasn't — rotating costs nothing and takes minutes.

[ ] **Meta WhatsApp access token** — will be replaced anyway in Step 3 with a permanent one, so no separate action needed here.

[ ] **Cloudinary API secret** — Cloudinary Console > Settings > Access Keys > regenerate. Paste the new value into your `.env` (and later, your hosting provider's environment variables).

[ ] **Google service-account key** — Google Cloud Console > IAM & Admin > Service Accounts > find the bot's account > Keys > delete the old key > Add Key > create new JSON key. Then run: `node scripts/apply-google-credentials.js <path-to-new-key.json>` from the project folder.

[ ] **ngrok authtoken** — dashboard.ngrok.com > Authtokens > revoke the old one, generate a new one (or simply stop using ngrok once you're deployed — see Phase D).

[ ] **WEBHOOK_VERIFY_TOKEN** — currently a weak, guessable value. Generate a strong one (command below) and use the new value everywhere it's needed (your `.env` AND the Meta webhook config in Step 8).

Generate a strong random token (run in your project terminal):

```
node -e "console.log(require('crypto').randomBytes(24).toString('hex'))"
```

Run this twice — once for `WEBHOOK_VERIFY_TOKEN`, once for `READINESS_SECRET` (used in Step 5). Save both outputs somewhere safe; you'll paste them into your hosting provider's environment variables.

> ***Why this matters:*** *a leaked access token lets someone send WhatsApp messages as your business (reputation risk + cost); a leaked service-account key gives read/write access to your entire ticket and customer database; a guessable verify token lets someone fake Meta's handshake.*

Phase B — Get Real Credentials from Meta

STEP 3 — Generate a permanent WhatsApp access token — FREE

Your current token is **temporary** and expires in roughly 24 hours (sometimes sooner). A permanent System User token never expires unless you revoke it.

- Go to **business.facebook.com** > Business Settings (gear icon).
- **Users > System Users > Add** > name it (e.g. “chanakya-bot”) > Role: **Admin** > Create.
- Click the new system user > **Add Assets** > select **WhatsApp Accounts** > pick your Chanakya WhatsApp Business Account > enable **Manage WhatsApp Business Account** > Save.
- Click **Generate New Token** > pick your Chanakya app > **Token Expiration: Never**.
- Permissions: tick **whatsapp_business_messaging** and **whatsapp_business_management**.
- Click **Generate** and **copy the token immediately** — Meta shows it only once.
- Paste it as the new `META_ACCESS_TOKEN` value (local `.env` now; hosting provider in Phase C).
- Verify it works: run `npm run verify:meta` from the project folder.

STEP 4 — Get your Meta App Secret — CRITICAL

This is currently **empty** in your configuration. Without it, the bot cannot verify that inbound webhook requests genuinely come from Meta — in production the code will refuse to start until this is set (by design, so this gap can't accidentally ship).

- Meta App Dashboard > your app > **Settings > Basic**.
- Copy the **App Secret** value (click “Show”).
- Set it as `META_APP_SECRET` in your hosting provider's environment variables.

STEP 5 — Complete business verification — FREE

Unverified WhatsApp Business accounts are capped at 250 conversations per rolling 24-hour period. Verified businesses start at 1,000/day and the limit auto-increases with good usage history. Start this early — Meta's review can take 1–3 business days.

- Meta Business Manager > Business Settings > **Security Center** (or “Business verification”).
- Submit your business documents (registration, address proof, etc. as requested).
- Separately, get your **display name** (“Chanakya – The Bag Studio”) approved under WhatsApp > API Setup > Business Profile.

Phase C — Deploy the Bot

Recommendation from our hosting comparison: start on **Railway's free 30-day trial** ($5 credit, no card needed) to confirm everything works end-to-end, then move to **Railway Hobby ($5/month, ~Rs. 420)** before the trial credit runs out, so the bot never goes dark on a live customer number.

STEP 6 — **Create the Railway project and connect GitHub** — Rs. 0 (trial) then $5/mo

- Go to **railway.app** > sign up / log in (GitHub login is easiest).
- **New Project > Deploy from GitHub repo** > select your `chanakya-bot` repo.
- Railway auto-detects `railway.json` (already in the repo) and configures the build/start commands.

STEP 7 — **Set every environment variable on Railway** — CRITICAL

In your Railway project > **Variables** tab, add everything from your local `.env`, with these specific values:

[ ] `NODE_ENV=production` — turns on webhook signature enforcement, fail-fast startup checks, and locks down the /ready endpoint.

[ ] `META_ACCESS_TOKEN` = the permanent token from Step 3.

[ ] `META_APP_SECRET` = the value from Step 4.

[ ] `WEBHOOK_VERIFY_TOKEN` = the new random value from Step 2.

[ ] `READINESS_SECRET` = the second random value from Step 2.

[ ] `META_PHONE_NUMBER_ID`, `GOOGLE_SHEETS_ID`, `GOOGLE_SERVICE_ACCOUNT_EMAIL`, `GOOGLE_PRIVATE_KEY` (rotated in Step 2), `CLOUDINARY_CLOUD_NAME/API_KEY/API_SECRET` (rotated in Step 2), `OWNER_PHONE_VEDANT`, `OWNER_PHONE_VATSAL`.

[ ] `TRUST_PROXY=1` — tells the bot it's behind Railway's proxy, so rate-limiting reads the real visitor IP.

[ ] Do **NOT** set `SKIP_WEBHOOK_SIGNATURE` at all.

Optional cost-control tuning (safe to leave at defaults — see the companion cost PDF for what these mean): `STATUS_PUSH_MODE` (default `smart`), `PICKUP_REMINDER_ENABLED`, `REASSURANCE_ENABLED` (both default off), `OUTBOUND_KILL_SWITCH` (see Appendix).

STEP 8 — **Attach a persistent volume** — CRITICAL

Railway's filesystem is wiped on every redeploy by default. The bot stores two small files under `data/`: which tickets it has already notified about, and when each customer last messaged (used to know if a reply is free). Without a volume, every redeploy silently resets both — harmless, but it means a status change during the deploy window could be missed, and everyone briefly looks like a “new” contact for the free-message check.

- In Railway > your service > **Settings > Volumes > New Volume**.
- Mount path: `/data` (or any path you prefer).
- Add variable `REPAIR_STATUS_CACHE_PATH=/data/repair_status_snapshot.json`.
- Add variable `LAST_CONTACT_CACHE_PATH=/data/last_contact.json`.

STEP 9 Deploy and verify it's healthy

FREE

- Railway deploys automatically once variables are saved. Watch the **Deploy Logs** tab.
- You should see the startup banner and `[META] Startup check OK`.
- Railway gives you a public URL like `https://chanakya-bot-production.up.railway.app`.
- Visit `<that-url>/health` in a browser — expect `{"ok":true,...}`.
- Visit `<that-url>/ready?secret=YOUR_READINESS_SECRET` — expect `"whatsapp_credentials":"ok"`.

Phase D — Connect Meta to Your Live Bot

STEP 10	Point Meta's webhook at your Railway URL	CRITICAL

- Meta App Dashboard > **WhatsApp > Configuration**.
- **Webhook URL:** `https://YOUR-RAILWAY-URL/webhook`
- **Verify Token:** the same `WEBHOOK_VERIFY_TOKEN` value you set in Railway (Step 7).
- Click **Verify and Save**. Check Railway's logs for `[WEBHOOK] Verification successful`.
- Under **Webhook fields**, make sure **messages** is subscribed (toggled on).

ngrok is no longer needed after this point. *You can close it, and stop paying it any attention — the bot is reachable directly at your Railway URL.*

STEP 11	Add a payment method in Meta Business Manager	FREE to add

Meta requires a payment method on file to send messages beyond the small free testing tier, even though most of your traffic (customer replies) will be free. Business Settings > Payment methods > add a card. You will only be charged for messages that fall into a paid category — see the companion cost PDF.

Phase E — Sheet Refresh & Optional Templates

STEP 12 **Refresh the Google Sheet headers** FREE

A column (“last_reassurance_at”) was added to the repair-tickets sheet during hardening. Run this once from the project folder to make sure headers match:

```
node populate_sheet.js
```

This only writes header rows and the status dropdown — it does not touch existing ticket data.

STEP 13 **(Optional) Create the pickup notification template** **~Rs. 0.13/message when used**

The bot already sends free status updates whenever a customer has messaged it in the last ~24 hours. For the one case where a customer needs to be proactively told their bag is ready *and* they haven't messaged recently, you can optionally enable a paid template so that message still reaches them reliably. This is optional — skip it and the bot still works; that one message just won't be force-delivered to silent customers.

- Meta > WhatsApp Manager > **Message Templates > Create Template**.
- Category: **Utility**.
- Body text, exactly two variables: e.g. *“Your repair {{1}} is ready for pickup at {{2}}. We hold bags free for 30 days.”*
- Submit for approval in English, Hindi, and Gujarati (usually approved within minutes to a few hours).
- Once approved, set `PICKUP_TEMPLATE_NAME` in Railway to the template's exact name and redeploy.

Phase F — Smoke Test on the Real Number

Do this full pass yourself before telling any customer the number exists. Budget about 10 minutes.

[ ] Send “hi” > confirm the welcome menu appears with buttons.

[ ] Tap **Repair My Bag** > complete the flow including sending a real photo > confirm you get a ticket ID and the owner alert arrives on Vedant/Vatsal's phone.

[ ] Tap **Track My Repair** > confirm your own ticket(s) show up in the picker with live status — and confirm a *different* phone number cannot see your ticket.

[ ] Try the **Corporate / Bulk** flow end-to-end.

[ ] Try **Store Locations** — confirm the map link opens correctly.

[ ] Type **terms** — confirm the summary (and PDF, if configured) arrives.

[ ] Type **STOP** > confirm the opt-out confirmation message, then check the Google Sheet's `opt_in_contacts` tab — column D should flip to FALSE.

[ ] Type **RESUME** > confirm re-opt-in works.

[ ] In the Google Sheet, manually change a ticket's status column > within 15 minutes confirm you get a free status push (since you just messaged, your window is open).

STEP 14 **Go live**

Once every item above checks out: add the WhatsApp number to your website, Google Business Profile, and any in-store signage. You're live.

Appendix — The Kill Switch (Emergency Stop)

If the bot ever misbehaves in production — a bug causes a message loop, a cron job misfires, or something looks wrong — you have one variable that instantly silences all outbound messages without touching code or redeploying anything else.

How to use it:

[ ] In Railway > Variables, set `OUTBOUND_KILL_SWITCH=1`.

[ ] Railway redeploys automatically (usually under a minute). Once live, the bot stops sending anything billable.

[ ] Investigate the issue calmly — the bot keeps receiving and reading messages, it just won't reply.

[ ] When resolved, delete the variable (or set it to `0`) and let it redeploy. Sending resumes immediately.

It works alongside two automatic limits that need no action from you: a per-minute and per-day cap on outbound messages (catches a runaway loop within seconds, even while you're asleep), and a cap on how many people a single broadcast can reach. The kill switch is your manual override on top of those.

Appendix — Quick Reference: Where Things Live

Thing	Where
Full launch checklist (technical detail)	LAUNCH_CHECKLIST.md in the project
Environment variable reference	.env.example in the project
Health check	https://YOUR-URL/health
Readiness / credential check	https://YOUR-URL/ready?secret=...
Emergency stop	OUTBOUND_KILL_SWITCH=1 in Railway Variables
Repo	GitHub (private) — via GitHub Desktop